

AL-AZHARI LIBRARY Backend — Phase 0 Implementation Report

Status: Phase 0 (Foundation Only) Successfully Completed

Repository Branch: `main`

Working Directory: `Back-end`

1. Files Created & Updated

Foundation Infrastructure & Configuration

- Back-end/src/app.ts — Express application factory with security headers (Helmet), CORS, JSON/URL-encoded parsers (1MB limit), request ID middleware, Pino HTTP logging, rate limiting, `/health/live`, `/health/ready`, versioned `/api/v1` base path mounting, 404 handler, and centralized error handling.
- Back-end/src/server.ts — Server bootstrap with immediate liveness activation, asynchronous MongoDB connection for readiness checks, Socket.IO attachment, and graceful shutdown on `SIGTERM` / `SIGINT`.
- Back-end/src/config/env.ts — Strict Zod environment schema with production safety refinements (rejecting localhost DBs, non-secure cookies, and dev default secrets in production), safe error summary formatting (no secret leaking), and testable `validateEnv()` helper.
- Back-end/src/config/logger.ts — Pino structured logger with comprehensive redaction of sensitive paths (passwords, tokens, payment proof URLs, secrets, cookies, customer notes) and silent mode during automated tests.
- Back-end/src/common/http/status-codes.ts — Standard HTTP status codes mapping.
- Back-end/src/common/http/envelope.ts — Standard response envelope formatting utilities (`formatSuccessResponse` , `formatErrorResponse` , `sendSuccessResponse` , `sendErrorResponse`).
- Back-end/src/common/http/index.ts — Re-exports for HTTP common utilities.
- Back-end/src/common/errors/AppError.ts — Base `AppError` and specialized error classes (`ValidationError` , `BadRequestError` , `UnauthorizedError` , `ForbiddenError` , `NotFoundError` , `ConflictError` , `BusinessRuleViolationError` , `DependencyUnavailableError`).
- Back-end/src/common/errors/errorCodes.ts — Machine-readable error codes constants (`VALIDATION_ERROR` , `AUTH_REQUIRED` , `NOT_FOUND` , `RESOURCE_CONFLICT` , `ORDER_STATE_CONFLICT` , `INTERNAL_ERROR` , etc.).
- Back-end/src/common/errors/index.ts — Unified error module exports.
- Back-end/src/common/middleware/error.middleware.ts — Centralized error handler managing `AppError` , Zod validation errors, JSON syntax parsing errors, and unexpected server errors (guaranteeing zero internal stack trace exposure).
- Back-end/src/common/middleware/request-id.middleware.ts — Cryptographically secure `crypto.randomUUID()` request correlation identifier, input sanitization, and `x-request-id` response header propagation.
- Back-end/src/common/types/response.ts — TypeScript interfaces for consistent API envelopes (`ApiResponseSuccess` , `ApiResponseError` , `ApiResponse`).
- Back-end/src/common/utis/response.util.ts — Response envelope helpers injecting `requestId` at root and in `meta` .

Test Suites

- Back-end/tests/unit/env.test.ts — Unit tests for environment schema validation, invalid input rejection, and production constraint enforcement.
- Back-end/tests/unit/errors.test.ts — Unit tests for `AppError` class hierarchy, default HTTP statuses, and error codes.
- Back-end/tests/unit/response.test.ts — Unit tests verifying response formatting structures and pagination metadata.
- Back-end/tests/integration/health.test.ts — Integration tests for `/health/live` (process liveness without DB), `/health/ready` (dependency readiness), and health summaries.

- `Back-end/tests/integration/app-bootstrap.test.ts` — Integration tests for application bootstrap and base path health routing.
- `Back-end/tests/api/app.test.ts` — End-to-end API tests for request ID generation/preservation, 404 error envelope, malformed JSON handling, known application errors, unexpected internal errors (no stack trace leak), Helmet headers, and CORS.

Documentation

- `Back-end/docs/ARCHITECTURE.md` — Modular monolith overview, layered architecture, module directory responsibilities, and request lifecycle.
- `Back-end/docs/ERROR-HANDLING.md` — Error handling architecture, standard error envelope, error codes table, class hierarchy, and middleware behavior.
- `Back-end/docs/ENVIRONMENT.md` — Environment variables reference, validation rules, production safety invariants, and secret redaction.
- `Back-end/docs/PHASE-0-FOUNDATION.md` — Detailed Phase 0 deliverables, scope restrictions (what is intentionally unbuilt), and running instructions.
- `Back-end/README.md` — Updated quickstart guide, scripts, and documentation cross-links.

2. Dependencies

All dependencies adhere strictly to the foundation requirements without unnecessary infrastructure:

- **Runtime:** `express`, `zod`, `pino`, `pino-http`, `helmet`, `cors`, `cookie-parser`, `express-rate-limit`, `mongoose`, `socket.io`, `dotenv`
- **Development:** `typescript`, `tsx`, `jest`, `ts-jest`, `supertest`, `@types/node`, `@types/express`, `@types/cors`, `@types/supertest`, `@types/jest`, `eslint`, `prettier`
- **Explicitly Excluded:** Redis, BullMQ, Atlas Search, payment gateways, and cloud SDK business logic (per the implementation plan).

3. Scripts Added / Updated

Configured in `package.json` :

```
{
  "build": "tsc",
  "typecheck": "tsc --noEmit",
  "start": "node dist/server.js",
  "dev": "tsx watch src/server.ts",
  "test": "jest",
  "test:watch": "jest --watch",
  "lint": "eslint .",
  "lint:fix": "eslint . --fix",
  "format": "prettier --write \"src/**/*.ts\"",
  "format:check": "prettier --check \"src/**/*.ts\""
}
```

4. Foundation Architecture Implemented

1. **Modular Monolith Layout:** All 21 domain modules (`auth` , `users` , `addresses` , `products` , `categories` , `carts` , `orders` , `payments` , `inventory` , `services` , `quotations` , `preorders` , `returns` , `refunds` , `notifications` , `coupons` , `content` , `shipping` , `reports` , `audit` , `settings`) have their planned subdirectories (`controllers/` , `services/` , `repositories/` , `models/` , `schemas/` , `routes/` , `types/` , `index.ts`) scaffolded as empty placeholders for future phases.
2. **Versioned REST API Base Path:** Routes are mounted under `/api/v1` .
3. **Request ID:** Cryptographically secure (`crypto.randomUUID()`), attached to `req.id` , emitted in `X-Request-Id` response header, and included in success/error envelopes.
4. **Standard Envelopes:**
 - Success: `{ "success": true, "data": {...}, "requestId": "...", "meta": { "requestId": "...", "pagination": null, "timestamp": "..." } }`
 - Error: `{ "success": false, "error": { "code": "...", "message": "...", "details": null }, "requestId": "...", "meta": { "requestId": "...", "timestamp": "..." } }`
5. **Security & Logging:** Helmet headers, CORS policies with exposed `X-Request-Id` , 1MB payload limits, IP rate-limiting, and Pino log redaction of sensitive credentials.
6. **Health Probes:**
 - `GET /health/live` — Verifies process liveness without requiring MongoDB (returns HTTP 200 { `status: "ok"` }).
 - `GET /health/ready` — Explicitly reports database connection readiness (returns HTTP 200 if connected, HTTP 503 `DEPENDENCY_UNAVAILABLE` if disconnected).

5. Verification & Test Results

All commands executed successfully with zero errors:

Command	Status	Result Summary
<code>npm run typecheck</code>	PASS	<code>tsc --noEmit</code> completed with 0 type errors
<code>npm run lint</code>	PASS	ESLint verified <code>src</code> and <code>tests</code> with 0 errors and 0 warnings
<code>npm run build</code>	PASS	Clean production build compiled into <code>dist/</code> with source maps
<code>npm test</code>	PASS	6 test suites passed, 34 total tests passed (100% pass rate)
<code>npm run dev</code>	PASS	Server boots via <code>tsx watch</code> , responds to <code>/health/live</code> with HTTP 200 and request ID, responds to unknown routes with HTTP 404 error envelope, and performs graceful shutdown on <code>SIGTERM</code>

6. Remaining Issues & Next Phase Handoff

- **Remaining Issues:** None. Foundation is clean and stable.
- **Scope Restriction Maintained:** No business logic, auth endpoints, catalog models, cart operations, checkout workflows, email/Cloudinary integrations, or background workers were implemented.
- **Ready for Phase 1:** Configuration & Database connectivity (Mongoose models, migrations, and validators).